

DevOps Daily presents

git blame

Production is down. Everyone has a theory.

The Pod That Wouldn't Die

Chapter 1 · Volume 1: Everything Is Fine

Preview edition



Published by DevOps Daily · devops-daily.com

Friday, 4:57 PM.

Sam

It's literally a two-line
change.

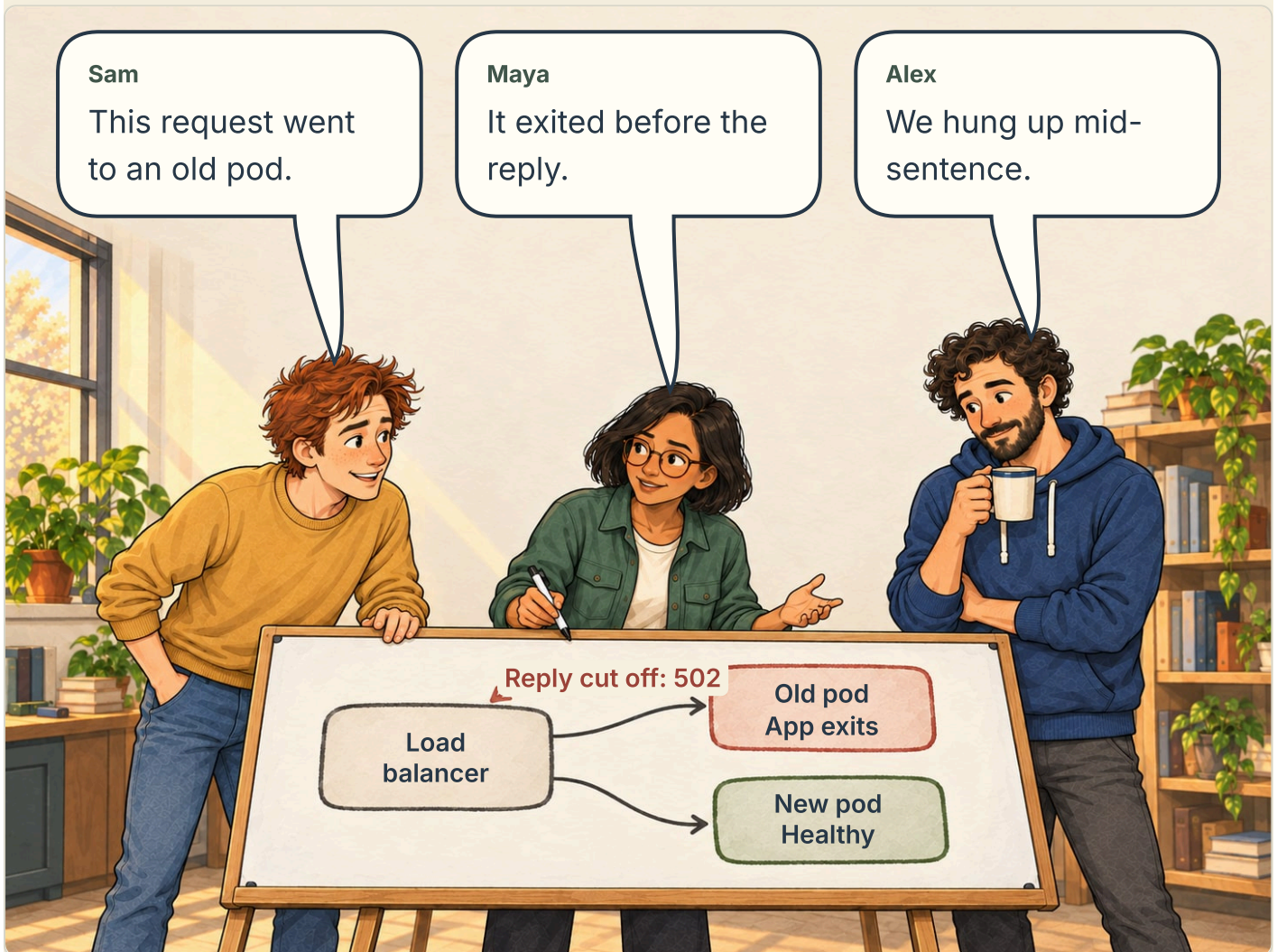
Maya

That's what you said last
Friday.









The two-line change gets a slightly larger companion.



Test environment first. A watched production rollout afterward.



Sam

Even the slow request finished.

Alex

Excellent. Boring graphs.

Chris

One more tiny change before we go?

Chris

Monday.



Under the Hood

The new pods were healthy. The failed request was still waiting for an old pod to reply.

During the rollout, the old checkout app received SIGTERM and exited immediately. Its connection closed before the response finished, and the load balancer returned a 502. A replacement pod cannot take over a request already running elsewhere.

The team made shutdown a handoff: allow traffic withdrawal, stop accepting new connections, finish accepted requests, close dependencies, then exit within a bounded deadline. They tested requests overlapping retirement, including a deliberately slow request. For the first corrected release, they used healthy parallel replicas and kept old processes alive until routing had moved and their in-flight work finished. New code did not repair old running pods.

- Readiness affects admission of traffic; it does not finish accepted work or instantly update every load balancer. Replacement readiness and departure draining are separate checks.
- A preStop hook normally runs before the container's stop signal and consumes the same terminationGracePeriodSeconds budget. A measured delay can allow propagation; sleeping alone does not drain requests.
- In this Node.js 22 HTTP/1.1 example, `server.close()` stops accepting new connections while active requests finish. Closing active connections forcefully is not the normal first step, and detached work or database transactions need their own handling.
- The load-balancer allowance, application deadline and pod grace budget must fit real request bounds. Kubernetes can forcibly kill remaining processes when grace expires. Other frameworks, signals, proxies and protocols require their own review.

A lost checkout response does not prove the order failed to commit. Safe retries of writes require an appropriate idempotency design. Shipt and its successful rollouts are fictional; a 502 has several possible causes, so correlate evidence before choosing a fix.

Read further

- [DevOps Daily: Deployments and ReplicaSets](#)
- [DevOps Daily: Services and Networking](#)
- [Kubernetes: Pod termination](#)
- [AWS: ALB 502 troubleshooting](#)